

Inteligência Artificial Na Prática — Fundamentos, Preparação E Análise De Dados

Gabrielly de Queiroz Pereira - gqpereira@uepg.br

Unidade 2: Fundamentos de Python para Análise de Dados e IA

AULA 2: FUNDAMENTOS DE PYTHON PARA ANÁLISE DE DADOS E IA

Introdução

A linguagem Python tem assumido um papel central no desenvolvimento de soluções voltadas à Análise de Dados e à Inteligência Artificial, tanto no ambiente acadêmico quanto no setor produtivo. Sua ampla adoção está relacionada à combinação entre simplicidade sintática, facilidade de aprendizagem e um extenso conjunto de bibliotecas especializadas que permitem desde a manipulação básica de dados até a implementação de modelos avançados de aprendizado de máquina. Conforme Mitchell (1997), sistemas capazes de aprender a partir de dados constituem o núcleo do aprendizado de máquina, e o uso de linguagens adequadas é essencial para viabilizar esses processos de forma eficiente.

No campo da ciência de dados, a programação é uma habilidade fundamental para a organização, exploração e interpretação de grandes volumes de informações. Han, Kamber e Pei (2012) afirmam que a mineração de dados envolve etapas como preparação, transformação e análise dos dados, etapas essas que dependem diretamente de ferramentas computacionais robustas. Nesse contexto, o Python destaca-se por permitir a implementação prática desses processos de forma acessível e altamente produtiva. Além disso, Provost e Fawcett (2016) ressaltam que a análise orientada por dados exige não apenas ferramentas tecnológicas, mas também compreensão dos princípios computacionais que sustentam a extração de conhecimento a partir dos dados.

Dessa forma, esta aula tem como finalidade apresentar os fundamentos da linguagem Python aplicados à Análise de Dados e à Inteligência Artificial, oferecendo ao estudante as bases necessárias de programação para a manipulação de dados, construção de rotinas computacionais e compreensão dos processos que sustentam aplicações modernas nessas áreas.

Desenvolvimento

1. Ambiente de desenvolvimento: Google Colab e Visual Studio Code

O primeiro passo para trabalhar com Python aplicado à Análise de Dados é a definição do ambiente de desenvolvimento. Atualmente, duas das ferramentas mais

utilizadas são o Google Colab e o Visual Studio Code (VS Code), cada uma com finalidades e vantagens específicas.

O Google Colab é uma plataforma online que permite programar em Python diretamente no navegador, sem necessidade de instalação local. Basta acessar o site:

<https://colab.research.google.com>

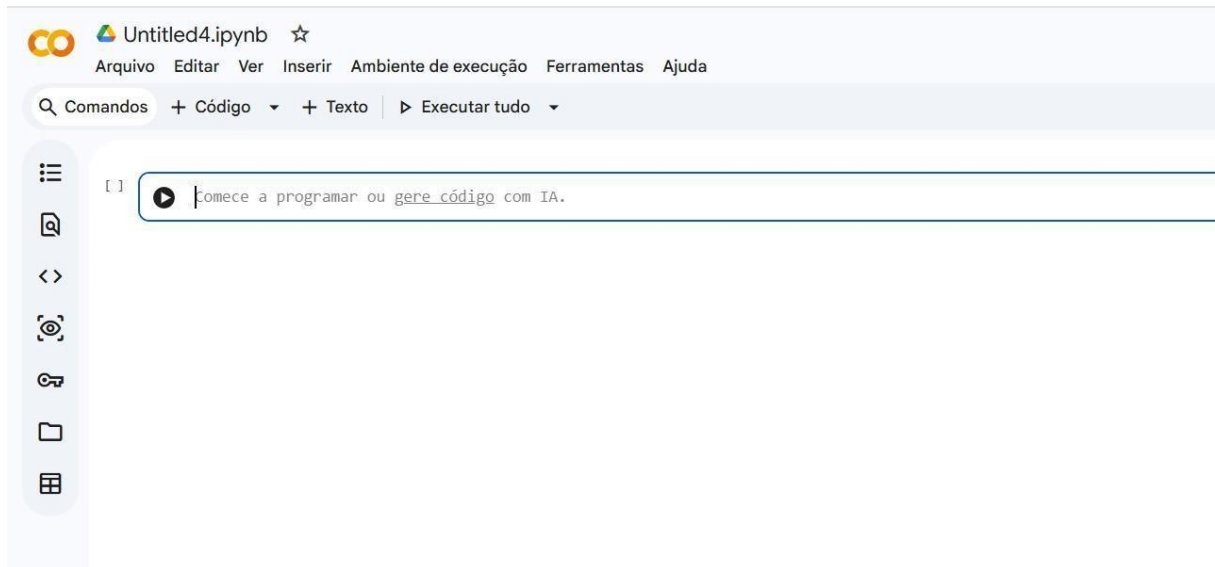


Imagem 1 - Tela inicial do notebook do Colab

O usuário pode criar um novo notebook, escrever códigos em blocos e executá-los diretamente. O Colab já vem com as principais bibliotecas de ciência de dados instaladas, como NumPy, Pandas e Matplotlib, o que facilita muito o início dos estudos.

Exemplo simples no Colab:

```
print("Olá, mundo!")
```

Esse comando exibe uma mensagem na tela e serve como primeiro contato com a linguagem.

Já o Visual Studio Code (VS Code) é um editor de código que funciona localmente no computador. Ele deve ser baixado no site oficial:

<https://code.visualstudio.com>

Após a instalação, também é necessário ter o Python instalado no computador, disponível em:

<https://www.python.org>

No VS Code, é possível criar arquivos com extensão .py, escrever códigos, instalar bibliotecas e organizar projetos completos, sendo muito utilizado em ambientes profissionais.

2. Instalação de bibliotecas e gerenciamento de dependências

No Python, as bibliotecas são instaladas por meio do pip, que é o gerenciador de pacotes da linguagem. No Google Colab, normalmente não é necessário instalar manualmente as principais bibliotecas. Já no VS Code, a instalação é feita pelo terminal com comandos como:

```
pip install numpy
```

```
pip install
```

```
pandas
```

Esses comandos fazem o download e a instalação das bibliotecas no ambiente local.

As dependências são as bibliotecas externas das quais um projeto precisa para funcionar corretamente. Em projetos maiores, costuma-se registrar essas dependências em um arquivo chamado requirements.txt.

3. Introdução ao Python: tipos de dados, estruturas e funções básicas

Python é uma linguagem interpretada, simples e de fácil leitura. Um dos primeiros conceitos fundamentais envolve os tipos de dados, que definem o tipo de informação armazenada.

Exemplos de tipos básicos:

```
idade = 20      # inteiro (int)
```

```
altura = 1.65   # número decimal
```

```
(float) nome = "Ana"      # texto
```

```
(string)
```

```
ativo = True     # lógico (boolean)
```

As estruturas de decisão permitem que o programa tome decisões:

```
if idade >= 18:
```

```
    print("Maior de idade")
```

else:

```
print("Menor de idade")
```

As estruturas de repetição permitem executar um bloco várias vezes:

```
for i in range(5):
```

```
    print(i)
```

As funções básicas organizam o código em blocos reutilizáveis:

```
def soma(a, b):
```

```
    return a + b
```

```
resultado = soma(5, 3)
```

```
print(resultado)
```

Essa função recebe dois valores, soma e retorna o resultado.

4. Biblioteca NumPy: computação numérica

A biblioteca NumPy é utilizada principalmente para trabalhar com vetores, matrizes e operações matemáticas de forma eficiente. É amplamente usada em aprendizado de máquina e processamento de dados.

Importação da

biblioteca: `import numpy`

as `np`

Exemplo de criação de um vetor:

```
valores = np.array([10, 20, 30, 40])
```

```
print(valores)
```

Exemplo de cálculo da média:

```
media = np.mean(valores)
```

```
print(media)
```

Esse tipo de operação é fundamental em análises estatísticas e algoritmos de IA.

5. Biblioteca Pandas: manipulação e análise de dados

A biblioteca Pandas é a principal ferramenta para trabalhar com dados em tabelas, muito semelhante às planilhas do Excel. Ela permite ler arquivos, organizar dados, filtrar informações e realizar análises.

Importação da biblioteca:

```
import pandas as pd
```

Exemplo de criação de uma tabela (DataFrame):

```
dados = pd.DataFrame({  
    "Nome": ["Ana", "João", "Maria"],  
    "Idade": [20, 22, 19]  
})
```

```
print(dados)
```

Exemplo de leitura de arquivo CSV:

```
df = pd.read_csv("dados.csv")
```

```
print(df.head())
```

Nesse exemplo, o arquivo dados.csv é lido e as cinco primeiras linhas são exibidas.

Pandas é essencial para:

- Limpeza de dados
- Análise exploratória
- Preparação de dados para modelos de IA

6. Aplicação prática na Análise de Dados e na IA

Com os fundamentos apresentados nesta aula, o estudante já passa a compreender como:

- Configurar seu ambiente de trabalho
- Escrever os primeiros códigos em Python
- Manipular dados numéricos com NumPy
- Organizar e analisar dados com Pandas

Essas ferramentas formam a base necessária para avançar, nas próximas aulas, para temas como visualização de dados, aprendizado de máquina e aplicações em inteligência artificial.

NumPy: Base Numérica do Python Científico

NumPy é a principal biblioteca para cálculos numéricos em Python. Seu núcleo é o *ndarray*, um vetor ou matriz multidimensional que oferece velocidade muito maior que listas comuns de Python. Em Machine Learning, ele é usado para operações vetorizadas, criação de matrizes, manipulação de tensores e cálculos estatísticos rápidos.

Entre as funções mais utilizadas estão:

- `sum()` – soma dos valores de um array, usada para cálculos de totais, erros acumulados e normalizações.

- `mean()` – cálculo da média, essencial para medidas de tendência central e escalonamento de dados.
- `median()` – cálculo da mediana, útil para análises robustas em presença de outliers.
- `std()` – desvio padrão, frequentemente aplicado em padronização (z-score) e análise de dispersão.
- `min()` e `max()` – valores mínimo e máximo, usados para normalização e identificação de limites de variáveis.
- `argmax()` e `argmin()` – posições dos maiores e menores valores, aplicados na escolha da melhor ação, melhor predição ou maior probabilidade.
- `reshape()` – reorganização de arrays, essencial para preparar dados na forma exigida por modelos.
- `linspace()` e `arange()` – geração de sequências numéricas, úteis em testes e distribuição de pontos.

NumPy é também amplamente utilizado para implementações internas de frameworks de ML, pois suas operações vetorizadas são otimizadas em C, garantindo alto desempenho.

Pandas: Estruturação e Manipulação de Dados

Pandas fornece duas estruturas principais: `Series` (vetores) e `DataFrame` (tabelas). É a biblioteca mais usada para análise e limpeza de dados em Machine Learning, pois permite lidar com planilhas completas de forma prática e eficiente.

As funções e métodos mais comuns incluem:

- `head()` e `tail()` – visualização rápida das primeiras ou últimas linhas de uma tabela.
- `info()` – resumo geral do dataset, mostrando tipos, contagem de valores e presença de nulos.
- `describe()` – resumo estatístico (média, mediana, mínimo, máximo, quartis), usado como primeira exploração dos dados.

- `sum()` – soma de colunas, útil em análises agregadas e verificações de consistência.
- `mean()`, `median()` e `std()` – estatísticas de tendência central e dispersão por coluna.
- `value_counts()` – contagem de categorias, muito útil em análises de classificação.
- `isna()` e `fillna()` – detecção e preenchimento de valores ausentes, etapa essencial do pré-processamento.
- `groupby()` – agregações por categorias, aplicadas para entender comportamentos específicos de cada grupo.
- `merge()` e `concat()` – junção de tabelas, indispensável quando os dados estão distribuídos em vários arquivos.
- `drop()` – remoção de colunas ou linhas que não são úteis.
- `sort_values()` – ordenação baseada em alguma variável importante.

Aplicações Diretas em Machine Learning

As funções e operações numéricas ou tabulares servem de base para várias etapas essenciais do pipeline:

1. Limpeza de dados – remoção de nulos, inconsistências e duplicatas.
2. Análise exploratória – cálculo de médias, medianas, desvios, correlações e contagens.
3. Pré-processamento – normalização, padronização e criação de novas variáveis.
4. Preparação do dataset – seleção de colunas, junção de tabelas e divisão entre treino e teste.
5. Transformações matemáticas – escalonamento, vetorização, manipulação de matrizes e tensores.

NumPy e Pandas, portanto, constituem o alicerce técnico da maior parte dos projetos de Machine Learning, permitindo que os dados passem por todas as fases de preparação com clareza, velocidade e segurança.

Exemplos com NumPy

```
import numpy as np
```

```
# Criando um array NumPy simples
```

```
arr = np.array([10, 20, 30, 40, 50])
```

```
print("Array:", arr)
```

```
# Soma dos elementos
```

```
print("Soma:",
```

```
arr.sum())
```

```
# Média
```

```
print("Média:", arr.mean())
```

```
# Mediana
```

```
print("Mediana:", np.median(arr))
```

```
# Desvio padrão
```

```
print("Desvio padrão:", arr.std())
```

```
# Mínimo e máximo
```

```
print("Mínimo:", arr.min())
```

```
print("Máximo:", arr.max())
```

```
# Índices dos valores mínimo e máximo
```

```
print("Posição do máximo:", arr.argmax())
```

```
print("Posição do mínimo:", arr.argmin())
```

Exemplo NumPy com matrizes e reshape()

```
# Criando um array sequencial de 0 a 11
```

```
arr2 = np.arange(12)
```

```
# Mudando o formato para 3 linhas e 4 colunas
```

```
matriz = arr2.reshape(3, 4)
```

```
print("Matriz 3x4:\n", matriz)
```

```
# Soma dos elementos por linha
```

```
print("Soma por linha:", matriz.sum(axis=1))
```

```
# Soma por coluna
```

```
print("Soma por coluna:", matriz.sum(axis=0))
```

Geração de dados com linspace() e arange()

```
# linspace gera valores igualmente espaçados
```



```
lin = np.linspace(0, 1, 5)
```

```
print("Linspace:", lin)
```

```
# arange cria sequência com passo definido
```

```
ar = np.arange(0, 10, 2)
```

```
print("Arange:", ar)
```

Exemplos com Pandas

```
import pandas as pd
```

```
# Criando um DataFrame
```

```
dados = pd.DataFrame({  
    "aluno": ["A", "B", "C", "D", "E"],  
    "nota": [8.5, 7.0, 9.2, 6.5, 8.0],  
    "horas_estudo": [5, 3, 8, 2, 4]  
})
```

```
print(dados)
```

Estatísticas básicas

```
# Primeiras linhas
```

```
print(dados.head())
```



Resumo das colunas

```
print(dados.info())
```

Resumo estatístico

```
print(dados.describe())
```

Usando sum(), mean(), median() e std()

```
print("Soma das notas:", dados["nota"].sum())
```

```
print("Média das notas:", dados["nota"].mean())
```

```
print("Mediana das notas:", dados["nota"].median())
```

```
print("Desvio padrão das notas:", dados["nota"].std())
```

Contagem de categorias com value_counts()

```
dados["turma"] = ["A", "A", "B", "B", "A"]
```

```
print(dados["turma"].value_counts())
```

Detectando e preenchendo valores ausentes

```
dados2 = dados.copy()
```

```
dados2.loc[2, "nota"] = None # criando um valor ausente
```

```
print("Valores ausentes:")
```

```
print(dados2.isna())
```

Preenchendo com a média

```
dados2["nota"] = dados2["nota"].fillna(dados2["nota"].mean())
```

```
print("Após
```

```
preenchimento:")
```

```
print(dados2)
```

Ordenação e filtragem

Ordenar por nota

```
print(dados.sort_values("nota",
```

```
ascending=False))
```

Filtrar alunos com nota acima de 8

```
print(dados[dados["nota"] > 8])
```

Agrupamento com groupby()

```
grupo = dados.groupby("turma")["nota"].mean()
```

```
print("Média de notas por turma:")
```

```
print(grupo)
```

Junção de DataFrames (merge)

```
outra_tabela = pd.DataFrame({
```

```
    "aluno": ["A", "B", "C", "D", "E"],
```

```
"faltas": [2, 0, 1, 3, 0]  
})
```

```
merged = dados.merge(outra_tabela, on="aluno")  
print(merged)
```

Concatenação de tabelas

```
nova_linha = pd.DataFrame({"aluno": ["F"], "nota": [7.8], "horas_estudo": [3], "turma":  
["B"]})  
  
concatenado = pd.concat([dados, nova_linha], ignore_index=True)  
print(concatenado)
```

Referências:

DAVENPORT, Thomas H.; RONANKI, Rajeev. Artificial intelligence for the real world. **Harvard Business Review**, v. 96, n. 1, p. 108–116, 2018.

HAN, Jiawei; KAMBER, Micheline; PEI, Jian. **Data Mining: Concepts and Techniques**. 3. ed. Waltham: Morgan Kaufmann, 2012.

MANYIKA, James et al. **Artificial Intelligence, Automation, and the Economy**. Washington: McKinsey Global Institute, 2017.

MITCHELL, Tom M. **Machine Learning**. New York: McGraw-Hill, 1997.

PROVOST, Foster; FAWCETT, Tom. **Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking**. Sebastopol: O'Reilly Media, 2016.

RUSSELL, Stuart; NORVIG, Peter. **Inteligência Artificial**. 3. ed. Rio de Janeiro: Elsevier, 201

